Next-Token Recommendation for Decentralized-Exchange Traders

A project plan

Gergely Madarász Levente Tóth Béla Jakab

October 2026

Budapest University of Technology and Economics
Advanced Machine Learning, project plan

The question in one sentence

Given what a trader has bought and sold so far,
which **new** token will it trade in the **next two days**?

What this talk covers

1. The setting, explained from zero
2. What the literature says
3. The data
4. Preprocessing and the train/dev/test split
5. The baseline, the metric and the protocol
6. The proposed method and why it fits
7. Work plan, risks, limitations

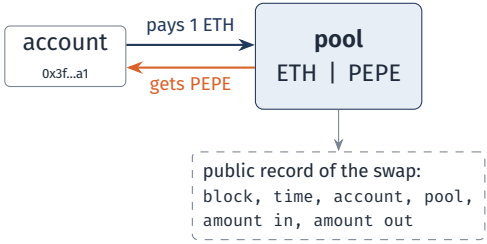
What is done and what is not

- ✅ done: data of 14 days, preprocessing, split, a fixed baseline with results
- 🕒 planned: more data (12 weeks), the models, the test
- No model results are claimed in this talk.

The setting, from zero

Four words we need: token, exchange, pool, swap

- **Token:** a digital asset on the Ethereum blockchain. There are tens of thousands (e.g. USDC, LINK, PEPE).
- **Decentralized exchange (DEX):** a program on the blockchain where anyone can trade tokens. We use Uniswap V2/V3 and copies of it.
- **Pool:** a reserve of *two* tokens. You put one token in and take the other out.
- **Swap:** one such trade. Every swap is recorded publicly, with the **account** (address) that signed it.



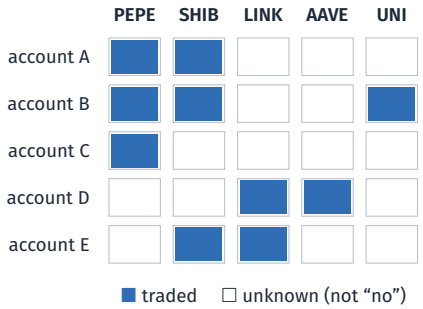
Takeaway. Every trade of every account is public. This gives us a large, free log of who traded what, and when.

This is a recommendation problem

A recommender ranks *items* for each *user*, like films on Netflix.

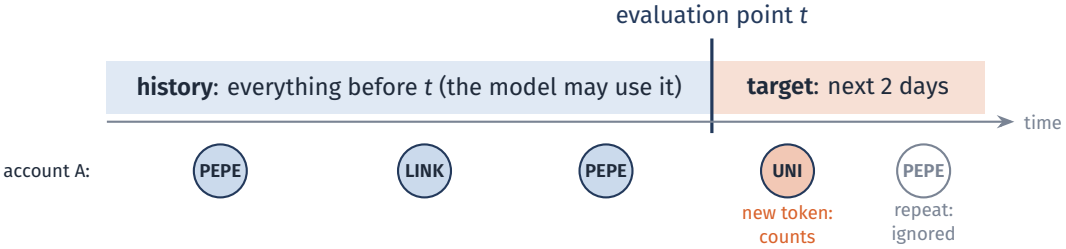
	Netflix	Our project
user	viewer	trading account
item	film	token
signal	watched	traded

Implicit feedback: we only see what an account *did* trade. Not trading a token does not mean dislike; we have no ratings and no negatives [Hu 2008, Rendle 2009].



Toy example. In the real data only 0.014% of the cells are filled.

What exactly is predicted: one evaluation point



1. At time t, for every account, rank all tokens it has **never** traded before t.
2. Show the top 10.
3. A **hit** is a token in the list that the account trades for the first time in the two days after t.

Takeaway. The labels come from the blockchain itself: exact, free, and available for any period.

What the literature says

The literature in four groups (21 references)

1. Recommending from implicit feedback

Weighted matrix factorization [Hu 2008]; ranking loss [Rendle 2009]; **item-based nearest neighbours** [Sarwar 2001]; top-*N* evaluation [Cremonesi 2010]; linear item model [Steck 2019]; short sessions [Hidasi 2016].

2. Simple methods are hard to beat

Many neural recommenders lose to tuned simple ones [Dacrema 2019, Rendle 2020, Rendle 2021, Anelli 2022, Ludewig 2018]; popularity is a strong baseline when measured at the right time [Li 2021, Ji 2020a].

3. Evaluating without fooling yourself

Per-user splits leak the future [Ji 2020b]; the split changes conclusions [Meng 2020]; sampled metrics mislead [Krichene 2020]; new items need content [Schein 2002].

4. Ranking and on-chain assets

Learning to rank [Burges 2010]; two-stage systems [Covington 2016]; NFT recommenders [Choi 2024, Li 2023].

🔍 **Gap:** in our searches we found *no* paper recommending fungible tokens from on-chain trades. (Absence in a few searches, not proof.)

Read in full: [Sarwar 2001, Cremonesi 2010, Schein 2002]. The others at abstract level or more.

Three papers read in full, and what we take from each

Paper	Main finding	What we use
Sarwar et al. [Sarwar 2001]	Score an item by its similarity to the user's own items. Keeping only 25–50 neighbours per item keeps 96–98% of the accuracy.	The core method; a short neighbour list per token.
Cremonesi et al. [Cremonesi 2010]	For top-*N* lists, a plain "most popular" list is close to sophisticated models; on the less popular items it collapses.	Popularity as the serious baseline; results also without the most popular tokens.
Schein et al. [Schein 2002]	Items nobody has used yet cannot be found by collaborative filtering; they need item content, and trivial heuristics must be compared.	Cold start as a separate problem, compared with simple heuristics.

Details and what does *not* carry over: backup slides.

How the literature changed the plan

Lesson	Consequence for us	Source
Short histories favour item-to-item methods	Item nearest neighbours first	[Sarwar 2001, Ludewig 2018]
Simple tuned models match complex ones	No neural network in the plan	[Dacrema 2019, Anelli 2022]
Popularity is strong and often mis-measured	Popularity is the fixed baseline, measured at time t	[Li 2021, Ji 2020a]
Per-user splits leak the future	One global timeline for all users	[Ji 2020b]
Sampled metrics mislead	Every model ranks *all* tokens	[Krichene 2020]
Repeat and new items are different tasks	Only new tokens are the target	[Li 2021]
Generate candidates, then rank them	Two-stage method with a learned ranker	[Covington 2016, Burges 2010]

The data

The dataset of the feasibility study

done

14 days	2.71 M	211,547	20,014	440 MB
24 Sep – 8 Oct 2026	swaps	accounts	tokens	on disk

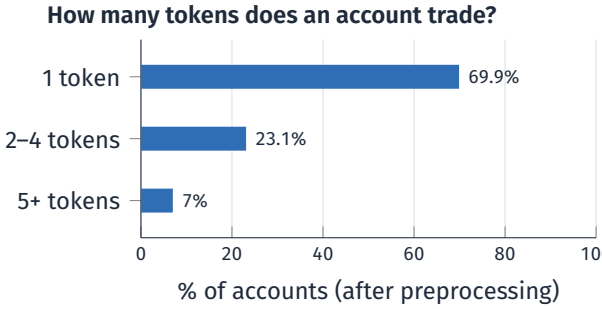
Source. Every swap on Uniswap V2/V3-style pools on Ethereum, read from free public endpoints. No key, no licence restriction.

One row per swap: block, time, signing account, pool, amounts. The signs of the amounts tell what was paid and what was received.

Validation.

- no duplicate events, no missing values
- 99.1% of blocks contain a swap
- tokens known for 99.98% of swaps
- 164k–225k swaps on each full day

The data is very sparse and very skewed



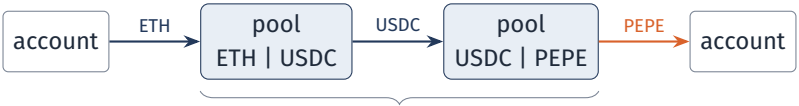
157,060 accounts: 47,280 with ≥2 tokens, 11,000 with ≥5.

- The account–token table is **0.014%** full.
- The **typical account trades one token**.
- The 10 most used pools carry **29%** of all swaps.
- The 1,000 most active accounts make **57%** of all swaps (many are bots).

Takeaway. Many events, but few accounts with enough history to personalize for. Popularity will be a strong reference.

Preprocessing and split

The central decision: the item is the token, not the pool



- 20% of transactions pass through more than one pool.
- Of the “new pools” an account touches, only 73% are a token new to it.

So: each transaction is reduced to what the account finally *sold* (ETH) and *bought* (PEPE). USDC was only passed through and is dropped.

Takeaway. The trader chooses the token; software chooses the pools. We model the trader’s choice.

Other preprocessing decisions

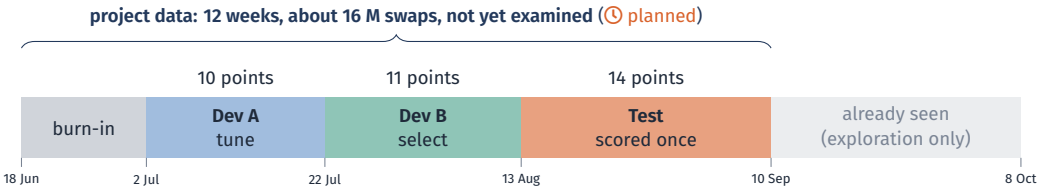
done

What we found in the data	What we do
Base currencies are in almost every trade (WETH 81% of accounts, USDC 40%, USDT 29%; next token: 5%)	Remove tokens traded by >10% of accounts. Decided on the training period only.
Arbitrage bots trade in cycles that end where they start	Such cycles leave no bought or sold token and drop out by themselves.
A few accounts make thousands of swaps	Interactions are binary (traded or not); every metric is an average over accounts.
Half of next trades are repeats	Only *new* tokens are the target.
Some target tokens did not exist yet at time t	Kept, and counted as misses. Scores reflect the real task.

Result: 157,060 accounts, 19,631 tokens, median 1 token per account.

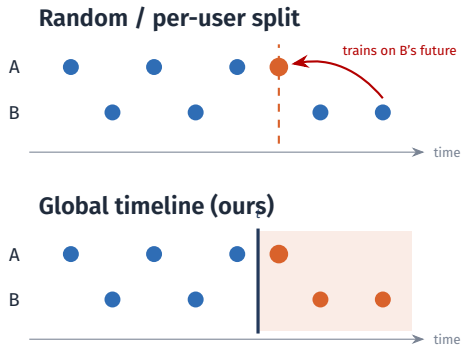
The split: one global timeline

Rule. At an evaluation point *t*, *all* accounts share the same cut. Points are grouped into periods by calendar date.



- Each point: history = everything before it; target = the next 2 days. Targets do **not overlap**.
- Order is chronological: tune → select → test. Test is scored **once**, from a frozen commit.
- Honest caveat: the test period is untouched, but it lies *before* the data we explored.

Why not a random split? Leakage



Leakage = the model sees information it could not have had at prediction time [Ji 2020b].

Main controls

- one cut in time for all accounts
- token statistics only from the history
- base-currency list fixed on training days
- settings on Dev A, choice on Dev B, test once
- ranker validated on held-out *points*

Full list of 13 risks and controls: backup slides.

Baseline, metric and protocol

The metric: recall@10, on a small example

	list for account A
1	MOG
2	UNI ✔
3	LINK
4	ONDO
5	PEPE2
6	AAVE
7	ENA
8	SPX
9	LDO
10	TURBO

A actually picks up

- UNI
- WOJAK
- NEWCOIN

(launched after t: unreachable)

Token names: illustration only.

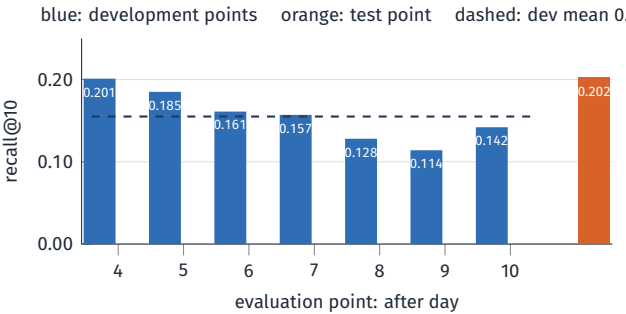
$$\text{recall@10}(A) = \frac{|\text{list} \cap \text{picked up}|}{|\text{picked up}|} = \frac{1}{3}$$

Then average over all evaluated accounts.

- **Why 10:** a person can read a short list.
- **Why average over accounts:** a bot with 10,000 swaps counts as much as a person with 2.
- **Also reported:** NDCG@10 (rewards hits near the top), hit rate (share of accounts with ≥1 hit), coverage (share of tokens ever shown).

The fixed baseline: “the 10 most popular tokens”

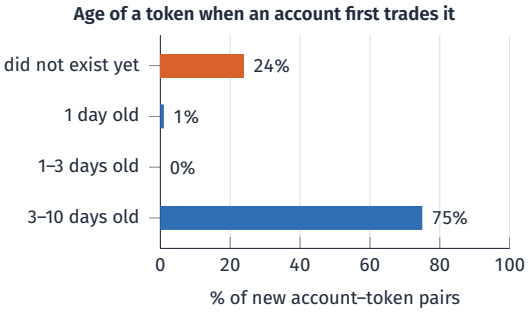
Popularity: rank tokens by how many distinct accounts traded them in the history; skip the account’s own. No settings; the same list for everyone; measured at time t [Ji 2020a].



- Finds **about 1 in 6** of the new tokens.
- At least one right token for **1 account in 4**.
- Moves between **0.11 and 0.20** by day; sampling noise is only ±0.007.
- Shows **under 1%** of all tokens to anyone.

Takeaway. The day matters more than the sample. One evaluation point is not enough to compare two models.

What the baseline reveals about the task



The two days after day 10, 24,583 pairs.

1. Cold start. 15–25% of the target tokens did not exist at time *t*. No model of past trades can list them.

2. Bimodal adoption. Tokens are picked up at launch, or once established; little in between.

3. Weakest where personalization should help. Baseline recall@10: 0.18 for accounts with 1–4 tokens, 0.12 for accounts with 5 or more.

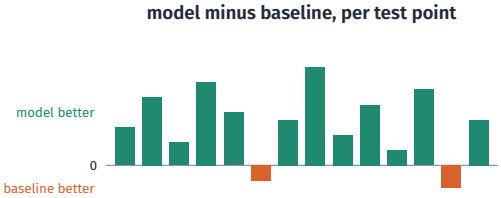
Evaluation protocol and the success criterion

At every evaluation point

1. fit every model on the history only;
2. for each account, score *all* tokens, remove its own, keep 10;
3. score the lists against the next 2 days.

Then compare with the baseline, paired by point:

- mean difference, bootstrap interval over points;
- points won: 12 of 14 needed (sign test, 5%);
- also by history size, and without the most popular tokens [Cremonesi 2010].



Schematic only. No model results exist yet.

Takeaway. Fixed in advance: success = higher mean recall@10 than the baseline on the 14 test points, with a bootstrap interval of the paired difference that excludes zero.

The proposed method

Stage 1: item-based nearest neighbours [Sarwar 2001]

Idea: two tokens are similar if the *same accounts* trade them. Recommend tokens similar to the account's own.

U_i = accounts that traded token i; H_u = tokens of account u.

$$s_{ij} = \frac{|U_i \cap U_j|}{\sqrt{|U_i|\,|U_j|} + \lambda} \qquad r_{ui} = \sum_{j \in H_u} s_{ji}$$

- λ (shrinkage) distrusts tokens with few accounts.
- Keep only the k most similar tokens per token.
- Works from **one token** of history.

Toy example (5 accounts, $\lambda = 0$)

	PEPE	SHIB	LINK	AAVE
A	■	■		
B	■	■		
C	■	?	?	?
D			■	■
E		■	■	

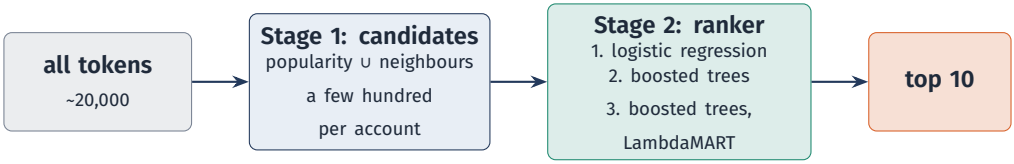
Account C has only PEPE.

$$s_{\text{PEPE,SHIB}} = \frac{2}{\sqrt{3 \cdot 3}} = 0.67 \qquad s_{\text{PEPE,LINK}} = 0$$

$\Rightarrow$ recommend **SHIB** to C.

Takeaway. Combined with popularity as $w\,\tilde{p}_i + (1 - w)\,\tilde{r}_{ui}$. With $w = 1$ this *is* the baseline, so the baseline is a special case.

Stage 2: a learned ranker on top [Covington 2016, Burges 2010]



One training row = (point, account, candidate token). Label 1 if the account trades it for the first time in the next two days.

Features (all from the history): popularity at several time scales, neighbour scores, account history size, token age and early activity.

Guarding against overfitting

- simplest model first
- trained on Dev A only (10 points)
- validated on held-out points
- compared with the best *fixed* combination, not only the baseline

Why this method fits the task and the data

Property of the task	How the method answers
Only positives, no ratings	Binary data, co-usage similarity, ranking metrics [Hu 2008, Rendle 2009]
The typical account has one token	Item neighbours need only one item; user-specific factors would be fitted on almost nothing [Hidasi 2016, Ludewig 2018]
Popularity is strong	Popularity is a term of the score [Li 2021]
Thousands of tokens with a handful of accounts	Shrinkage λ and top-k truncation
~20k tokens, ~100k accounts	Sparse linear algebra on one machine; nothing heavy needed [Dacrema 2019, Anelli 2022]
The best mix of signals may differ between accounts	A learned ranker combines them [Burges 2010]
Exact labels at every point	Each dev point is a labelled training set

The three core problems of the project

1. Short histories

The typical account has one token.

Question: what do neighbours add to popularity?

Work: tune k, λ, w; report by history size (1, 2–4, 5+).

2. Time weighting

Interest moves fast; the baseline swings 0.11–0.20.

Question: how much should old trades count?

Work: weight a trade of age a by $2^{-a/h}$; tune the half-life h.

3. Cold start

15–25% of targets did not exist at time t.

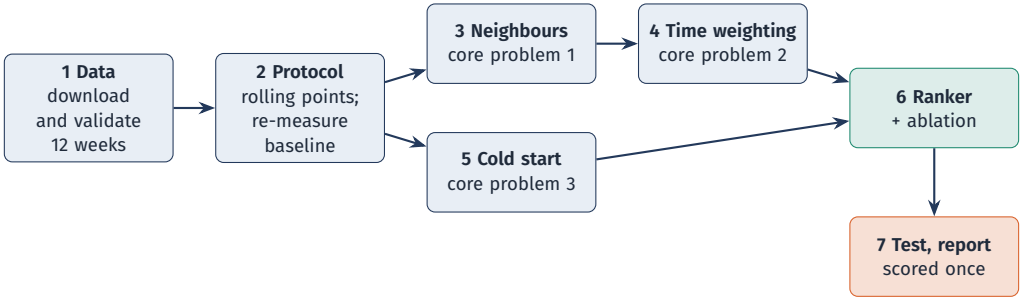
Question: how much is reachable with refreshed lists?

Work: rebuild lists every few hours; token features from the first hour [Schein 2002].

Takeaway. Each problem enters the ranker as a group of features. Removing one group at a time (ablation) measures what it contributes.

Work plan, risks, limitations

Work packages



- Every step is judged against the step before it, and always against the fixed baseline.
- Optional if time allows: matrix factorization (iALS) [Rendle 2021] and EASE [Steck 2019] as references.

Risks and limitations, stated plainly

Risks

- **The method may not beat popularity.** This is common in the literature [Li 2021]. A clean null result on 14 test points is still a result.
- **Day-to-day effects may hide model effects.** Hence many points and paired comparisons.
- **The ranker may overfit** 10 market situations. Hence simple models and held-out points.

Limitations of the evidence so far

- Two weeks, one market phase; only the baseline has results.
- One account ≠ one person; bots are not removed.
- Only Uniswap V2/V3-style pools.
- Most references were read at abstract level.

Use and harm. Recommending speculative tokens is not obviously good. We treat this as a study of trading behaviour and of evaluation, not as a product.

Summary

Task	For each account, rank the tokens it never traded; a hit is a first trade in the next 2 days.
Data	Uniswap-style swaps on Ethereum. Studied: 14 days, 2.71 M swaps. Planned: 12 earlier weeks.
Split	One global timeline. Dev A tunes, Dev B selects, 14 test points scored once.
Baseline	10 most popular tokens: recall@10 = 0.155 (0.11–0.20 by day).
Method	Item neighbours + popularity, then a learned ranker.
Core problems	Short histories, time weighting, cold start.
Success	Higher mean recall@10 on test, with a paired interval excluding zero.

Thank you. Questions?

References i

[Hu 2008] Y. Hu, Y. Koren, C. Volinsky. Collaborative filtering for implicit feedback datasets. *ICDM*, 2008.

[Rendle 2009] S. Rendle, C. Freudenthaler, Z. Gantner, L. Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. *UAI*, 2009. arXiv:1205.2618.

[Sarwar 2001] B. Sarwar, G. Karypis, J. Konstan, J. Riedl. Item-based collaborative filtering recommendation algorithms. *WWW*, 2001.

[Cremonesi 2010] P. Cremonesi, Y. Koren, R. Turrin. Performance of recommender algorithms on top-N recommendation tasks. *RecSys*, 2010.

[Steck 2019] H. Steck. Embarrassingly shallow autoencoders for sparse data. *WWW*, 2019. arXiv:1905.03375.

[Hidasi 2016] B. Hidasi, A. Karatzoglou, L. Baltrunas, D. Tikk. Session-based recommendations with recurrent neural networks. *ICLR*, 2016. arXiv:1511.06939.

[Dacrema 2019] M. F. Dacrema, P. Cremonesi, D. Jannach. Are we really making much progress? A worrying analysis of recent neural recommendation approaches. *RecSys*, 2019. arXiv:1907.06902.

[Rendle 2020] S. Rendle, W. Krichene, L. Zhang, J. Anderson. Neural collaborative filtering vs. matrix factorization revisited. *RecSys*, 2020. arXiv:2005.09683.

[Rendle 2021] S. Rendle, W. Krichene, L. Zhang, Y. Koren. Revisiting the performance of iALS on item recommendation benchmarks. arXiv:2110.14037, 2021.

References ii

[Anelli 2022] V. W. Anelli, A. Bellogín, T. Di Noia, D. Jannach, C. Pomo. Top-N recommendation algorithms: a quest for the state-of-the-art. *UMAP*, 2022. arXiv:2203.01155.

[Ludewig 2018] M. Ludewig, D. Jannach. Evaluation of session-based recommendation algorithms. *User Modeling and User-Adapted Interaction*, 2018. arXiv:1803.09587.

[Li 2021] M. Li, S. Jullien, M. Ariannezhad, M. de Rijke. A next basket recommendation reality check. arXiv:2109.14233, 2021.

[Ji 2020a] Y. Ji, A. Sun, J. Zhang, C. Li. A re-visit of the popularity baseline in recommender systems. *SIGIR*, 2020. arXiv:2005.13829.

[Ji 2020b] Y. Ji, A. Sun, J. Zhang, C. Li. A critical study on data leakage in recommender system offline evaluation. arXiv:2010.11060, 2020.

[Meng 2020] Z. Meng, R. McCreadie, C. Macdonald, I. Ounis. Exploring data splitting strategies for the evaluation of recommendation models. *RecSys*, 2020. arXiv:2007.13237.

[Krichene 2020] W. Krichene, S. Rendle. On sampled metrics for item recommendation. *KDD*, 2020.

[Schein 2002] A. Schein, A. Popescul, L. Ungar, D. Pennock. Methods and metrics for cold-start recommendations. *SIGIR*, 2002.

[Choi 2024] Choi et al. A recommender system for NFT collectibles with item feature. arXiv:2403.18305, 2024.

References iii

[Li 2023] S. Li, Y. Jin, P.-L. Hsu, Y.-S. Luo. NFT.mine: an xDeepFM-based recommender system for non-fungible token (NFT) buyers. arXiv:2306.03942, 2023.

[Burges 2010] C. J. C. Burges. From RankNet to LambdaRank to LambdaMART: an overview. Microsoft Research Tech. Report MSR-TR-2010-82, 2010.

[Covington 2016] P. Covington, J. Adams, E. Sargin. Deep neural networks for YouTube recommendations. *RecSys*, 2016.